Project Proposal: HyperLogLog

Ibrahim and Nika · Advanced Algorithms & Data Structures, Fall 2026 · 20 October 2026

1. Team and topic

We are Ibrahim and Nika, and our topic is HyperLogLog, the algorithm from our student seminar. It estimates how many different items are in a huge stream while using only a few kilobytes of memory. In this project we want to test how accurate it really is, and where it stops being accurate.

2. Idea and research questions

In the seminar we saw that HyperLogLog is very good for large counts, but that the basic version has trouble with small ones. We would like to see this ourselves and find out whether the fix is worth it.

Our research questions:

1. How does the error of HyperLogLog change with the true number of distinct items, and does linear counting (the small-count fix) remove the error for small counts?
2. How does the number of registers change the error, compared with the theory value $1.04/\sqrt{m}$?

What we expect

For large counts, the theory says the standard error is about $1.04/\sqrt{m}$, where m is the number of registers. For 16,384 registers that is 0.81%. We expect to measure roughly this, and it should not depend on how many items we count.

For small counts we expect trouble. With 16,384 registers and only 1,000 items, almost every register is still empty, so the basic formula should give about $0.72 \cdot 16{,}384 \approx 11{,}800$. That is around 12 times too big. Linear counting looks at how many registers are empty instead, and we expect it to be off by only about 1%.

Linear counting only changes the last step, not the update, so we do not expect it to make the algorithm slower.

We are not sure what happens between about 40,000 and 80,000 items, where the two formulas hand over to each other. If the error there is strange, we will report it and try to explain it.

One more thing: our slides say 1.5 KB in one place and 12 KB in another, and give a typical error of about 2%. With 16,384 registers of 6 bits the memory is 12 KB, and the error is 0.81%. We will check these numbers in the project and correct the slides if needed.

3. Implementation

We will write HyperLogLog ourselves, because we need to see the registers and count the empty ones. We write it in C++, because we both know it well and the leading-zero count is a

single built-in instruction there.

- Two versions in the same code: the basic estimator, and the basic estimator with linear counting when the estimate is at most 2.5m and some registers are empty.
- A 64-bit hash from an existing library: we use XXH3-64 from xxHash (header-only). The first p bits choose the register, the rest give the leading zeros.
- The number of registers is 2^p. We use p = 14 by default and try p from 4 to 16.
- Small tests to be sure it works: the example from slide 10, a count of 0, 1 and 2 items, and a check that merging two sketches gives the same result as one sketch of all items.

4. Evaluation plan

We keep the plan small on purpose, so that we can finish it well in four weeks.

What we compare with

The baseline is the basic HyperLogLog from the seminar. The exact answer comes from a normal hash set (std::unordered_set of 64-bit IDs), which also shows how much memory HyperLogLog saves.

What we measure

The main metric is the relative error, (estimate − true count) / true count. To see what we lose, we also measure the time per inserted item and the memory. For memory we report both what our code stores (1 byte per register) and the packed 6-bit size that the theory counts. To explain the error at small counts, we record the share of empty registers.

Inputs

We use random 64-bit IDs, with some of them repeated. The true count n goes from 10 to 10^7 on a log scale. If time remains in week 4, we add one real dataset: a public text file (for example a book from Project Gutenberg), where the IDs are the words and the true number of different words is counted with the hash set. It is an optional extra and the first thing we drop.

Experiments

#	On the x-axis	Curves	Fixed	What we plot
1	True count n	Basic HLL, HLL with linear counting	p = 14	Mean relative error and its spread
2	Number of registers (p)	Measured error, theory $1.04/\sqrt{m}$	n = 10^6	Spread of the error, and memory
3	True count n	Basic HLL, HLL with linear counting, hash set	p = 14	Time per item and memory

Randomness and timing

Every point is repeated with 20 random seeds, and all versions get the same seeds and inputs. We look at the distribution of the errors first, then report the mean with a 95% confidence interval and say on the plots what the error bars show. For the timing in experiment 3 we use a small benchmark harness of our own (Google Benchmark is not installed and we do not want to spend time on setting it up). It does a warm-up run, repeats each measurement several

times and reports the median and the spread; we build the keys outside the measured code, and we use every result so the compiler cannot remove the work. All runs are on one machine that is plugged in, with other programs closed.

5. Schedule

We have four weeks between the proposal and the defense. After the review we update this proposal within a few days.

Week	Ibrahim	Nika	Result
1	Hash set baseline, input generator	HyperLogLog code and tests	Working code, TBDs decided
2	Timing setup for experiment 3	Experiments 1 and 2	First plots
3	Experiment 3, memory	Compare results with our predictions	All main plots, list of surprises
4	Real dataset if we are on time	Report and slides together	Ready for the defense

If we fall behind, the real dataset is the first thing we drop.

Decisions (these were open before)

- **Language and benchmark tool:** C++ and our own small timing harness (see sections 3 and 4).
- **Hash library:** XXH3-64 from xxHash. It looked uniform on 10^6 random keys and also on 10^6 sequential keys (chi-square of the register buckets, distribution of the leading zeros, and balance of all 64 bits).
- **Real dataset:** optional, one public word list as described in section 4; skipped if there is no time.
- **Number of seeds:** 20. A first test run gave a spread of 0.79% at large counts against 0.81% in theory, so the intervals are narrow enough. We only add seeds if the intervals in the final plots turn out too wide.